The Evaluation Loop: How AI Systems Learn to Improve

Mick Cooney

12 October 2026

Table of Contents

1 The Core Idea: Measure It, Then Optimise It **3**
1.1 What This Primer Covers . . . 5
1.2 Reading Routes . . . 5
1.3 What This Primer Is Not . . . 6

Notation **6**

2 The Subjectivity Problem, Stated Plainly **7**

3 Manufacturing a Number: The Four Moves **9**
3.1 Decompose . . . 10
3.2 Compare . . . 10
3.3 Aggregate . . . 11
3.4 Automate . . . 12

4 Optional Companion: Evaluating Low-Volume Instruction and Workflow Wording **14**
4.1 What Changes, and What Does Not . . . 14
4.2 Quality Means Something Different Here . . . 14
4.3 The Lightweight Loop . . . 15
4.4 Worked Example: Improving a Discovery-First Workflow Instruction 16
4.5 Worked Example: Improving a Diagnostic Workflow Instruction Without Touching Weights . . . 17
4.6 A Reusable Micro-Protocol for Any Skill Wording Edit . . . 19
4.7 From Fuzzy Intuition to Controlled Iteration . . . 20
4.8 Companion Checklist: Evaluate and Improve One Workflow Instruction . . . 20
1. Identify the Failure Pattern . . . 20
2. Write Behavioural Anchors . . . 20
3. Identify the Ambiguous Phrase . . . 21
4. Rewrite for Observability . . . 21

5. A/B Test on Real Work . . . 21
6. Decide: Keep, Revert, or Iterate . . . 21
7. Log the Result . . . 21
8. Move to the next Skill or Repeat . . . 22
4.9 Companion Prompt Template: Copy and Paste to Evaluate a Workflow Instruction . . . 22
What You Can and Cannot Measure . . . 23
When the Primer's Machinery Returns . . . 23

5 Escaping Subjectivity: Verifiable Rewards 24

6 Closing the Loop: From Number Back to Model 26
6.1 Reward Model Training . . . 26
6.2 RLHF via PPO With a KL Leash . . . 27
6.3 Direct Preference Optimisation: Skipping the Reward Model . . . 27
6.4 Constitutional AI and RLAIF . . . 28
6.5 DSPy: Your Evaluation Metric as the Search Objective . . . 29
6.6 Continuous Integration and Deployment Evaluation Gates . . . 30
6.7 The Loop, Named . . . 30

7 The Bootstrapping Problem 31

8 Why the Number Lies: Goodhart and Its Family 32

9 Evaluating Agents, Not Just Outputs 34
9.1 Trajectory vs Outcome . . . 34
9.2 Credit Assignment at Scale . . . 35
9.3 SWE-bench and the Benchmark Trajectory . . . 35
9.4 Benchmark Gaming — Attacking the Evaluator . . . 36
9.5 What the METR Randomised Trial Found . . . 36
9.6 What This Means for the Loop . . . 36

10 How Frontier Labs Actually Run This 37
Anchor 1: Anthropic's RSP and the ASL Gate . . . 37
Anchor 2: OpenAI's Preparedness Framework . . . 38
Anchor 3: Chatbot Arena as Ecosystem-Level Evaluation . . . 38
What Running This Costs . . . 38

11 What Is Genuinely Unresolved 38
Goodhart's Treadmill Has No Exit . . . 39
Disagreement as Signal Versus Noise . . . 39
Benchmark Scores and Productivity Measure Different Outcomes . . . 39
Superhuman Oversight: The Hardest Problem . . . 39
A Brief Note on Test-Time Compute . . . 40
The Closing Motif . . . 40

12 Further Reading and Resources 40

This primer explains how evaluation turns model outputs into signals for improvement, from human comparisons to verifiable checks. It assumes basic familiarity with language models and software tests, but no prior knowledge of preference modelling or reinforcement learning. It is a guide to measurement choices and their limits, not a training manual or a current benchmark leaderboard.

Verified against primary sources for sections 1–3 and 5–11; unverified claims are flagged inline.

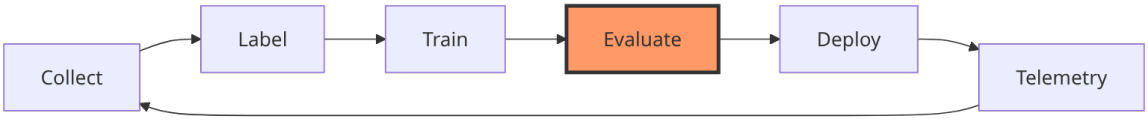
Benchmark scores, framework APIs, and vendor safety practices can change. Re-check the linked primary source and reproduce a measurement before using a historical score, leaderboard position, or API example for a current decision.

1 The Core Idea: Measure It, Then Optimise It

Here is the thing nobody tells you when you start building with AI: the model is often the easy part. You can download a strong open-weight model this afternoon, or call a frontier one over an API in three lines of code. What separates a system that gets better every month from one that plateaus on day one is whether you have built a loop that turns mistakes into the next improvement signal — and whether, somewhere in that loop, you can attach a defensible number or checklist to "how good was that?"

Andrej Karpathy gave this loop a clear articulation. In his 2017 essay "Software 2.0", he argued that for some problems, the program is shaped by data and optimisation rather than hand-written rules. Curating and labelling data then becomes central to software development. He identified a condition for that shift: "repeated evaluation is possible and cheap". This primer examines what happens when evaluation is difficult or costly.

By December 2022 Karpathy had sharpened the idea into what he called a data engine. His formulation, in paraphrase: competitive advantage in AI goes not so much to those with data, but to those with a data engine — iterated data acquisition, re-training, evaluation, deployment, and telemetry — and the winner is whoever can spin that loop fastest. The loop has a fixed shape:



You collect inputs, label them, train a model, evaluate it, deploy it, observe failures, and feed selected cases into the next collection round. Tesla describes using fleet data to develop and test driving features: in its account, Shadow Mode tests how a proposed feature would perform without activating it for the driver. That is an example of observing candidate behaviour before deployment. It does not mean every disagreement with a human driver is a labelled failure; deciding which cases matter still requires selection and review.

Karpathy calls this a "data engine". The phrase "evaluation loop" here emphasises the measurement step; it is not a claim that every workflow uses the same training and deployment cycle.

The organising principle is the distinction between writing a rule and evaluating a candidate result. Traditional programs encode many rules explicitly. Model training can search for behaviour when examples or a verifier provide a useful signal. You may be unable to write exhaustive rules for a good translation or explanation while still being able to compare candidates. The comparison must be reliable enough for the intended task; the following sections examine how to test that.

The evaluate step is the loop's sensor: it tests whether a change improved the outcome you care about. A score, comparison, or checklist can inform the next training or deployment decision. Training gradients can also come from objectives other than an evaluation score, and teams can deploy without a sound evaluation; the danger is that they then have weaker evidence about whether the change helped. Like a thermostat with a faulty thermometer, an optimisation loop can respond faithfully to a misleading signal.

So the deep problem of this primer is not only "how do we train models". It is the prior question: when the thing you care about is subjective — clarity, helpfulness, taste, judgment — how do you manufacture an evaluation signal trustworthy enough to optimise against? The main narrative answers it for model evaluation and post-training. A separate, optional companion applies the same measurement discipline to low-volume instruction and workflow wording, where a scalar training signal is the wrong tool.

1.1 What This Primer Covers

- The data engine loop and why evaluation is its load-bearing step.
- The main reading path: subjective measurement, verifiable rewards, and the post-training systems that optimise against those signals.
- Why open-ended quality resists measurement, and when reference-overlap metrics miss it.
- The generator-discriminator gap: why judging is easier than producing, and why that asymmetry rescues evaluation.
- The four moves that turn subjective quality into a usable score: Decompose, Compare, Aggregate, Automate.
- An optional companion on the lightweight instruction loop: rubric-driven transcript inspection for iterative workflow wording improvements.
- Behaviourally-anchored rubrics as a replacement for vague quality scales.
- Pairwise comparison and large-scale human preference collection (Chatbot Arena).
- Aggregating comparisons into latent ratings via Elo and the Bradley-Terry model, with the mathematics worked through.
- G-Eval and reading a judge's log-probabilities to recover a continuous score.
- The bridge from a fitted Bradley-Terry model to a reinforcement learning from human feedback (RLHF) reward model.
- Reinforcement Learning with Verifiable Rewards (RLVR), the SWE-bench trajectory, and the hard boundary where verifiable rewards stop helping.

1.2 Reading Routes

The main path runs from the subjectivity problem and the four moves through **Escaping Subjectivity: Verifiable Rewards** and **Closing the Loop: From Number Back to Model**, then continues through the limits and deployment sections. When the navigation reaches **Optional Companion: Evaluating Low-Volume Instruction and Workflow Wording**, skip directly to **Escaping Subjectivity: Verifiable Rewards** unless you are maintaining prompts or re-usable agent workflows from a small number of transcripts. Nothing in the main narrative depends on the companion.

1.3 What This Primer Is Not

There are no convergence theorems, gradient-descent derivations, or statistical guarantees of rating-system consistency here. The code snippets are illustrative kernels, not a deployable harness. Reward-model training is explained conceptually rather than implemented. The Further Reading section points to the detailed methods.

Notation

The mathematical sections use these symbols; each derivation also defines its variables where they first appear.

Symbol	Meaning	First Use
p_o, p_e, κ	Observed agreement, chance-expected agreement, and Cohen's chance-corrected agreement	Subjectivity Problem
R_A, R_B, E_A, S_A, K	Elo ratings, expected and observed scores for A, and update size	Aggregate
β_A, β_B	Bradley–Terry latent strengths of A and B	Aggregate
p_i, $\hat{s}$	Probability of score i and its expected value	Automate
n, c, k	Generated solutions, passing solutions, and selected attempts	Verifiable Rewards
x, y_w, y_l, π_θ, π_{ref}, β	Prompt, preferred and rejected responses, trained and reference policies, and constraint parameter	Direct Preference Optimisation

2 The Subjectivity Problem, Stated Plainly

Start with the question that actually stops you, the one with no obvious answer: if you want a system to do something well, how do you turn open-ended performance into a number you can optimise against? Optimisation needs a scalar. Gradient descent, hyperparameter sweeps, leaderboards, A/B tests — every one of them consumes a number and pushes it in a direction. But "did the model explain this well?" is not a number. It is a judgment, and judgments are exactly what we said modern AI is for: the things you can verify but cannot specify. The subjectivity problem is the gap between those two clauses.

Make it concrete. Suppose you pose a single question to a model:

> "What is a derivative contract? My manager mentioned it in a meeting and I have no finance background."

You get back two hypothetical replies. **Response Alpha** defines a derivative as a contract whose value depends on an underlying asset, then moves straight into notional value, delta hedging, and mark-to-market accounting. Those details may be correct, but they do not answer this reader at the requested level. **Response Beta** says: "A derivative is a contract whose value depends on something else, such as a share price. For example, an option can give you the right to buy a share later at a price agreed today. If the market price changes, that right may become more or less valuable." This is accessible and substantially accurate, though it describes one type of derivative rather than all of them. Which response better serves the stated reader? The hard part is defending that judgment with a measure.

The first thing people try is to compare the output against a reference answer. This is where the classical metrics live. **BLEU** (Bilingual Evaluation Understudy) was built for machine translation: it measures n-gram overlap between the candidate text and one or more reference translations, scored from 0 to 1. **ROUGE** (Recall-Oriented Understudy for Gisting Evaluation) was built for summarisation: it is recall-oriented, counting unigram and longest-common-subsequence overlap with reference summaries. Both ask the same underlying question — how many of the "right" words showed up in roughly the right arrangement?

For this derivative question, there can be several good explanations with different wording. **Response Beta** might overlap weakly with a reference written for finance specialists, despite answering the novice's question better. A random,

irrelevant answer could also overlap weakly. Word overlap alone cannot distinguish those cases or tell whether an explanation was understood. The exact BLEU or ROUGE ranking depends on the chosen reference and tokenisation; no scores have been computed for these hypothetical responses.

This is not a quirk of one example. On tasks that admit many valid phrasings, reference-overlap metrics can correlate poorly with human judgments; the G-Eval study reports this limitation for summarisation and dialogue. Translation and summarisation are not uniformly constrained: even a faithful translation can have several phrasings, and a summary can select different valid details. The relevant question is whether lexical overlap with the chosen references captures the quality you need to measure. For the beginner's explanation here, it does not: a precise overlap score can still measure the wrong quantity.

A scope note before going further: this primer treats bias and fairness only where they intersect measurement — where a scoring choice silently encodes a preference, or where a rater pool skews a rating. The broad ethics of fairness in AI is its own field and its own literature; here it appears strictly as a measurement concern.

When a reference does not capture all valid answers, pairwise comparison offers another source of evidence. In some tasks a reader can choose between two outputs more easily than they can produce a strong answer unaided. This **generator–discriminator gap** motivates preference comparisons, but it is not guaranteed: a non-expert may be unable to spot a subtle error in either candidate.

Human raters can disagree about the quality of the same response, especially when a scoring scale leaves "good" undefined. Before reporting agreement, specify the task, scale, rater pool, and whether the statistic measures identical labels or agreement after chance correction. Some disagreement reflects annotation noise; some reflects defensible differences in reader priorities. There is no single human agreement percentage that applies to every evaluation task.

Raw percent agreement can mislead. If two raters choose independently and uniformly from a 5-point scale, they match roughly 20% of the time by chance. Cohen's κ (Cohen, 1960) adjusts observed agreement for the chance agreement implied by the raters' label frequencies:

$$\kappa = \frac{p_o - p_e}{1 - p_e}.$$

Here p_o is observed agreement and p_e is chance-expected agreement calculated from the marginal label frequencies. A κ of 0 means no agreement beyond that baseline, and $\kappa = 1$ means exact agreement. Interpretation still depends on label prevalence, task, and consequences of a disagreement; a universal band such as "moderate" can conceal those differences.

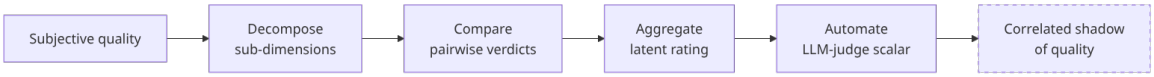
For more than two raters, Fleiss' κ and Krippendorff's α offer different ways to account for chance agreement and missing labels. Choose a statistic that matches the annotation design and report it alongside the raw distribution of judgments. A low statistic can expose an underspecified rubric, but it can also expose a real difference in preferences.

Automated judges must be calibrated against the task and raters whose preferences they are intended to approximate. Judge–human agreement and human–human agreement need the same items and statistic before they can be compared. Neither statistic is a universal ceiling on evaluation quality. A reported accuracy figure on a subjective task describes agreement with a particular set of labels, not access to an objective truth.

For the stated non-specialist audience, comparing **Response Alpha** and **Response Beta** may be easier to anchor than assigning an absolute score. Whether it yields more consistent judgments is an empirical question for the rater pool and rubric. The four moves below show how such comparisons can be turned into a training or selection signal.

3 Manufacturing a Number: The Four Moves

You cannot directly measure "quality". So you manufacture a stand-in. Four moves do the manufacturing: **Decompose, Compare, Aggregate, Automate**. Each move buys you something — tractability, reliability, a scalar, scale — and each one charges the same toll. What comes out the far end is never quality itself. It is a *correlated shadow of quality*: a number that moves with the thing you care about closely enough to optimise against, while never being the thing itself. Name that price at every step, because forgetting it is how teams end up optimising the shadow until it detaches from the substance.



3.1 Decompose

The first move attacks the vagueness head-on. "Was this a good explanation?" is hard to answer because "good" hides several distinct questions. So you break it into 3–6 sub-dimensions and give each one a small, **behaviourally-anchored** scale — every level defined by an observable behaviour rather than an adjective. Instead of "rate clarity 1–5", you write a rubric where a 4 means "fully resolves the task without any missing step" and a 0 means "leaves the core question unanswered". The anchors are behaviours, not vibes.

For the derivatives question, you might decompose "good answer" into accuracy, accessibility-to-a-non-specialist, completeness, and memorability. Now the two responses separate cleanly. **Response Alpha** scores high on accuracy, low on accessibility. **Response Beta** scores high on accessibility and memorability, slightly lower on accuracy. The vague tie has become four legible numbers. The price: a rubric is a theory of what matters, and the moment you write it down you have decided that these four dimensions, weighted this way, *are* quality. They are a correlated shadow of it — a good rubric tracks quality closely, but an answer can game every sub-dimension and still miss, because the rubric is a model of good, not good itself.

3.2 Compare

The second move cashes in the generator-discriminator gap. Stop scoring responses in isolation; show a judge two at a time and ask only which is better. This is what Chatbot Arena, developed by the Large Model Systems Organization (LMSYS) and UC Berkeley researchers, does at scale: users see two anonymous model responses to the same prompt and vote for the one they prefer. The 2024 Chatbot Arena paper reported more than 240,000 votes at publication. For our example, the comparison is exactly the question you could already answer: shown **Response Alpha** beside **Response Beta** for a self-declared non-specialist, which is better? The price: a pile of "A beat B" verdicts is useful evidence of preference, but it is not yet a number you can rank with. A heap of pairwise outcomes does not by itself define a scale.

3.3 Aggregate

The third move turns comparisons into a rating per item. Elo and Bradley–Terry both model pairwise outcomes, but use different estimation procedures and assumptions. **Elo**, borrowed from chess, assigns each item a rating and updates it after every match. Under its logistic expected-score model, if A and B have ratings R_A and R_B, A's expected score is:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}.$$

After a match, A's rating moves towards the observed score S_A:

$$R'_A = R_A + K(S_A - E_A).$$

Here K controls the update size; 32 is an illustrative choice rather than a universal setting. In code:

```python
def elo_update(rating_a: float, rating_b: float,
               outcome: float, k_factor: float = 32) -> tuple[float, float]:
    """
    outcome: 1.0 if A won, 0.5 if draw, 0.0 if B won.
    Returns updated (rating_a, rating_b).
    """
    expected_a = 1 / (1 + 10 ** ((rating_b - rating_a) / 400))
    expected_b = 1 - expected_a
    new_rating_a = rating_a + k_factor * (outcome - expected_a)
    new_rating_b = rating_b + k_factor * ((1 - outcome) - expected_b)
    return new_rating_a, new_rating_b
```

The **Bradley–Terry** model instead fits a latent strength β_A or β_B to each item. It models the probability that A beats B as:

$$P(A \succ B) = \frac{e^{\beta_A}}{e^{\beta_A} + e^{\beta_B}}.$$

The strengths are fitted by maximising the likelihood of the observed comparisons. If raters consistently prefer **Response Beta** for the stated audience, either model can give it a higher rating. That conclusion is conditional on the observed votes; the two responses in this primer have not been rated in a study.

The Bradley–Terry likelihood also motivates a common reward-model training objective. Instead of fitting one strength per fixed item, a reward model scores a response conditional on its prompt and is fitted to preference pairs. The analogy

is useful, but a leaderboard rating and a reward model differ in what they score, how they generalise, and how their estimates are used. §6 follows the training case. Both remain proxies for the preferences represented in their data.

3.4 Automate

The fourth move can reduce the number of human judgments needed for routine scoring. An LLM judge applies a rubric to candidates; a held-out human sample is still needed to check whether its scores reflect the intended readers' judgments. One implementation reads more than the judge's chosen token.

The prompt that does the asking looks like this:

```
You are an expert evaluator scoring how well a response answers a question
for a specific reader. Judge only the response shown; do not rewrite it.

QUESTION:
"What is a derivative contract? My manager mentioned it in a meeting and I
have no finance background."

RESPONSE:
{{response}}

Score the response on each dimension below, using the 1-5 scale:

  Accuracy       – is every claim correct and free of misleading shortcuts?
  Accessibility  – would a self-declared non-specialist follow it unaided?
  Completeness   – does it resolve the question without leaving a core gap?
  Memorability   – will the reader retain the core idea afterwards?

Scale anchors:
  5 – exemplary: meets the dimension with no reservation
  4 – strong: meets it with a minor, non-blocking flaw
  3 – adequate: meets it but with a flaw a careful reader would notice
  2 – weak: partially meets it; a real shortcoming on this dimension
  1 – failing: does not meet it at all

Return one integer score (1-5) per dimension, then one sentence of
justification per dimension. Output the scores first.
```

The chosen integer is the lossy part; G-Eval recovers the rest.

G-Eval (Liu et al., 2023) evaluates natural language generation (NLG) without taking the judge's output token "4" at face value. It reads the log-probabilities assigned to score tokens "1" through "5" and computes the expected value:

$$\widehat{s} = \sum_{i=1}^{5} i \cdot p_i.$$

Here p_i is the normalised probability assigned to score i. If the probabilities are

concentrated on 3 and 4, their expected value lies between them; a value such as 3.74 is illustrative, not a result from this example.

```python
import math

def g_eval_score(logprobs: dict[str, float]) -> float:
    """
    logprobs: mapping from score token ("1".."5") to raw log-prob
    Returns the expected score as a continuous float.
    """
    # Renormalise over the supplied score-token log-probabilities.
    total = sum(math.exp(lp) for lp in logprobs.values())
    p_scores = {token: math.exp(lp) / total for token, lp in logprobs.items()}
    expected_score = sum(int(token) * p for token, p in p_scores.items())
    return expected_score
```

The code assumes that `logprobs` contains all five intended score tokens from the same scoring position. It renormalises only those tokens; a production implementation must check tokenisation, missing mass, and the model interface. An LLM judge can then score the hypothetical responses without a human on every pass. Its score inherits the rubric's omissions and may add the judge's own biases. Check it against held-out human ratings for this task; the agreement statistics in §2 tell you how to interpret a measured comparison, not an assumed ceiling.

Two checks can expose judge bias. First, **swap the positions** of **Response Alpha** and **Response Beta** and compare the verdicts. A change in preference indicates order sensitivity; averaging scores does not necessarily cancel a nonlinear position effect. Second, ask **several independently selected judges** and examine where they disagree. Averaging may reduce independent noise, but shared training data or similar rubrics can preserve systematic bias. Both checks cost additional inference, so use held-out human judgments to decide whether they improve this evaluation.

These four moves are not specific to chatbots. Take an insurance company evaluating how well an LLM summarises complex policy documents for claimants. The moves apply identically: **Decompose** "good summary" into accuracy, coverage, plain-language score, and a missing-information penalty; **Compare** two candidate summaries pairwise; **Aggregate** the comparisons into a latent quality rating; **Automate** with an LLM judge that reads the policy and the summary together. Different domain, same machine — and the same toll, four times over: every number it produces is a correlated shadow of quality, never quality itself.

4 Optional Companion: Evaluating Low-Volume Instruction and Workflow Wording

This specialised companion is for practitioners who are editing instruction text rather than retraining model weights: a system prompt, a workflow definition, or a reusable agent instruction. Here, *skill* means only that last kind of reusable workflow instruction; it does not refer to a named feature of any particular agent framework. The model itself is fixed; the thing being changed is what it is told to do. This material is useful at low volume, but it is not a prerequisite for the main route, which resumes at **Escaping Subjectivity: Verifiable Rewards**.

Instruction editing also follows a change–observe–revise cycle, but its immediate evidence is often a small set of transcripts rather than enough independent comparisons to fit a rating model. Use the lightest measurement that answers the decision at hand, and add statistical machinery when the data and stakes justify it.

4.1 What Changes, and What Does Not

Model training changes parameters; instruction editing changes the context supplied to an otherwise fixed model. Those are different levers, although both can affect observed performance and neither guarantees a particular capability. A poor instruction can misdirect an otherwise capable model, while an instruction cannot reliably supply a capability absent from the model or its tools. Inspecting the same tasks under different wording helps distinguish these possibilities.

The practical consequence: you do not need a training signal to improve instructions. You need to be able to read the instruction's output and tell whether it did what you intended. That is a simpler requirement than manufacturing a gradient, but it is the same underlying demand from §2 — you must be able to evaluate the output before you can improve the system.

4.2 Quality Means Something Different Here

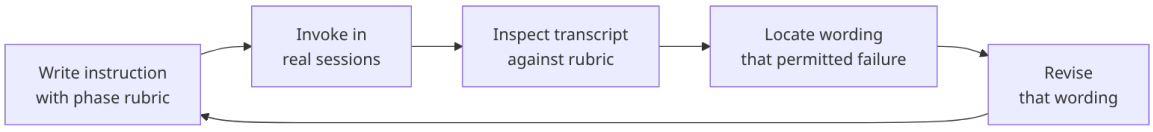
For the single-turn responses in earlier sections, "quality" meant something about the content of the answer — accuracy, accessibility, completeness. For instruction-following systems that prescribe a multi-step process, quality primarily means **adherence**: did the agent execute the intended steps, in the intended

order, without skipping? Adherence is observable. You do not need a judge to manufacture a number; you need a checklist that describes what correct execution looks like at each phase, and a session transcript to check it against. For a diagnostic skill that says *build a feedback loop before hypothesising*, the criterion is concrete: a runnable repro or failing test must exist in the session before any hypotheses are offered. Either it does or it does not. The generator-discriminator gap from §2 applies here too — it is much easier to check whether a phase was skipped than to produce the correct phase yourself — but the check costs almost nothing because the answer is binary.

This is the **Decompose** step from §3 applied to *process* quality. Instead of breaking "good answer" into accuracy, accessibility, and memorability, you break "correct skill execution" into its observable phases and write a behavioural anchor for each one: what does completion of this phase look like in the transcript? Write those anchors before you ever invoke the skill, because writing them forces you to notice which phase descriptions are vague enough to permit skipping. A rubric that says "phase 1 complete: a runnable repro exists" gives raters a visible test. A rubric that says "phase 1 complete: the agent seems to understand the problem" does not.

4.3 The Lightweight Loop

The loop that results is recognisably the data engine, run once per observed session or small batch:



The collect step is invoking the instruction in real work. The label step is checking the transcript against the rubric — did phase 1 complete before phase 2 began? The evaluate step is noting which phase the adherence failure clustered in. The revision step edits wording that may have contributed to the failure, then tests it again. Calling that step "training" is an analogy, not a parameter update. The loop runs at the speed of session review, with evaluation still informing the next decision.

A practical note on volume: fitted ratings need enough connected comparisons for useful uncertainty estimates; there is no universal minimum sample count.

A small collection of workflows invoked only occasionally may be better served by transcript inspection and explicit failure counts than by a leaderboard. G-Eval is a scoring method, not a rating estimator that "converges" with pairwise comparisons, so its calibration is a separate question.

4.4 Worked Example: Improving a Discovery-First Workflow Instruction

Take a workflow instruction that requires an agent to inspect repository evidence before asking the user questions. The goal is not to improve the model's latent reasoning ability; it is to reduce instruction ambiguity so execution is more consistent. The loop is the same shape as Karpathy's data engine, but each stage is lightweight and text-driven.

Suppose transcripts show a concrete failure pattern:

- The skill asks user questions too early, before enough repo exploration.
- The skill asks broad or duplicate questions that could have been answered from code.
- The skill claims "shared understanding" but still leaves unresolved design branches.

Now define behavioural anchors (your measurement surface):

Dimension	0 (Fail)	1 (Pass)
Exploration-first adherence	First user question appears before evidence summary	Evidence summary appears before any user question
Non-derivable questioning	Asks questions answerable from repo/docs	Asks only questions explicitly marked non-derivable
Tree closure	Ends interview with open branches	Replays decisions and lists no unresolved branches

With those anchors, rewrite only the wording most likely to permit failure. For example:

Before (weak, easy to misinterpret):

```
Ask one question at a time after exploration.
```

After (strong, testable):

```
Before asking the user a question, summarise the relevant repository evidence
and cite the files that support it.
Ask only questions whose answers you could not establish from that evidence;
for each question, explain what decision needs the answer.
Before finishing, replay all decisions and explicitly list unresolved branches
as "OPEN" (or state "No open branches").
```

The revision makes the evidence and question gate visible in a transcript. It does not prescribe an arbitrary number of facts or imply that repository search can be exhaustive.

Then run a small loop over real work (for example, 5 invocations):

1. Where possible, try versions A and B on matched tasks in random order; if only sequential sessions are available, record the task differences.
2. Score each session against the three binary anchors above.
3. Count pass rates per anchor for A vs B.
4. Treat a higher observed pass count as a reason for further testing, not proof that B caused the improvement.
5. If the pattern is unclear, collect more sessions or revise the ambiguous phrase and test again.

This is Karpathy's loop in miniature:

- Collect: session transcripts.
- Label: anchor pass/fail per session.
- Revise (analogous to training): edit instruction wording.
- Evaluate: compare anchor pass rates by wording version.
- Deploy: keep the better wording and continue observing.

No gradients, no reward model, no Elo required. The loop still qualifies as an evaluation engine because it converts fuzzy dissatisfaction ("this skill feels off") into a repeatable update rule over observed behaviour.

4.5 Worked Example: Improving a Diagnostic Workflow Instruction Without Touching Weights

The same method applies to a process-heavy diagnostic workflow instruction, where the central requirement is to establish a reproducible feedback loop before hypothesising. One possible failure is that the agent proposes causes before it has built a reproducible check. The transcript shows the sequence; it does not

by itself establish that wording, rather than missing tools or capability, caused it.

For a hypothetical set of five sessions, suppose you see this pattern:

- In three sessions, hypotheses appeared before any runnable failing test or deterministic repro script.
- In two sessions, the loop existed but was flaky (non-deterministic, noisy assertions).
- In one of those sessions, the agent moved to implementation after observing a single failing run rather than a stable repeated failure.

You can score this with an anchor set aimed at behavioural compliance:

Anchor	Fail condition	Pass condition
Repro-gate	Hypothesis appears before loop artefact exists	Runnable repro artefact exists before first hypothesis
Loop quality	Loop is flaky or symptom-level only	Loop is deterministic and asserts the target symptom
Transition discipline	Moves to fix without stable repro evidence	Moves to fix only after repeated stable failure signal

Now tighten only the instruction text that allowed the most common failure.

Before (high-level, permissive):

```
Do not proceed to Phase 2 until you have a loop you believe in.
```

After (operational, transcript-checkable):

```
Before any hypothesis is stated, produce a runnable repro artifact
(test, script, or command) and execute it twice with the same failing signal.
If the two runs do not fail identically, treat the loop as invalid and continue
loop-hardening; do not hypothesise yet.
Only enter fix implementation after a stable loop is present and the exact
assertion/symptom is captured in the artifact output.
```

This illustrative revision introduces a gate before hypothesising, asks for repeated evidence, and defines transition readiness. Two matching runs are a useful check for a deterministic repro, not a universal threshold: intermittent or environment-dependent faults may need a different observation plan.

Try a small comparison in live work; the counts are diagnostic, not a significance test:

1. Apply both wordings to matched tasks where possible, in random order; aim for at least three observations per condition as an initial inspection.
2. Score each session on the three anchors above.
3. Compute pass-rate deltas per anchor.
4. Inspect any gain in the repro gate and transition discipline alongside changes in loop quality before provisionally keeping the wording.

At this scale, avoid overfitting to one anecdote. Try the wording on tasks with different failure modes (for example, an API bug, a UI regression, and a performance issue), and inspect regressions before keeping it. Three tasks do not establish a general effect.

4.6 A Reusable Micro-Protocol for Any Skill Wording Edit

Changing one ambiguous phrase at a time makes the effect easier to inspect. The following protocol treats each edit as a testable hypothesis rather than a wholesale rewrite.

Use this protocol:

1. Pick one recurring failure in transcripts, phrased as behaviour, not vibes.
2. Write 2-4 binary anchors that would detect this failure.
3. Identify the narrowest phrase in the skill that could permit it.
4. Rewrite that phrase so pass/fail is visible in transcript evidence.
5. Test on 3-5 real invocations.
6. Inspect the anchor counts and task differences before provisionally keeping, revising, or reverting the edit; then log the result.

The following hypothetical record illustrates a one-line changelog; its deltas are not measured results from this primer:

```
workflow=diagnostic-repro | change=add repro hard gate | sample=6 sessions |
repro_gate +50pp | loop_quality +17pp | transition_discipline +33pp | keep
```

Those records form a small audit trail. Over time, they help you identify candidate patterns to test: which wording edits appear to improve adherence, which trade one failure for another, and which instructions may be stable enough to stop changing. Unmatched tasks can still confound those comparisons.

4.7 From Fuzzy Intuition to Controlled Iteration

For a workflow that "feels off", inspect whether the gate, criterion, or exit condition is observable. Each has a possible correction:

- Soft gate -> convert to explicit precondition with a visible artefact.
- Unobservable criterion -> replace adjectives with evidence requirements.
- Undefined exit -> add explicit completion language and unresolved-branch handling.

That is the practical payoff of importing Karpathy's loop into instruction design. You are not pretending to run frontier post-training on a handful of sessions. You are building a disciplined measurement-and-revision habit that is proportionate to the scale of skill files and still grounded in evaluation, not intuition alone.

4.8 Companion Checklist: Evaluate and Improve One Workflow Instruction

This is a reference checklist, not part of the main narrative. Use it to run a single improvement cycle on a workflow instruction. Copy it, fill it in, file the result, and repeat. Over time the one-line logs from step 7 become your instruction-wording audit trail.

1. Identify the Failure Pattern

- ☐ Review 3–5 recent transcripts of this skill.
- ☐ Describe the specific failure in behaviour terms (not "feels off" or "wrong", but "X appeared before Y in the transcript").
- ☐ Estimate frequency: does this failure appear in 1/5 sessions? 3/5? Every session?

2. Write Behavioural Anchors

- ☐ Define 2–4 binary criteria that would let you check if the failure occurred.
- ☐ For each criterion, write explicit pass and fail conditions.
- ☐ Verify: could you hand these criteria to someone unfamiliar with the skill and they could score a transcript?

3. Identify the Ambiguous Phrase

- □ Read the skill file and locate the phrase most likely to permit the failure.
- □ Is it a gate condition? A criterion description? An exit condition?
- □ Make sure it is one phrase, not a paragraph — you are editing a pinpoint, not a rewrite.

4. Rewrite for Observability

- □ Replace vague adjectives with evidence requirements appropriate to the task (e.g., “thorough exploration” -> “cite the relevant files and summarise what each establishes”).
- □ Add explicit gating language if this is a gate (“before X is done, never do Y”).
- □ Add explicit artefact requirements if this controls evidence (“produce a runnable test” not “understand the problem”).

5. A/B Test on Real Work

- □ Apply old and new wording to comparable tasks, randomising their order when practical.
- □ If sessions are sequential instead, record task and context differences.
- □ Score all sessions against the anchors from step 2.
- □ Record pass rate for each anchor under A and B.

6. Decide: Keep, Revert, or Iterate

- □ Compare each anchor on matched tasks and report its numerator and denominator.
- □ Inspect regressions and task differences; one or two sessions are not a calibrated noise threshold.
- □ Keep the change provisionally if the observed improvements justify another trial and no material regression appears.
- □ Otherwise revert, revise the phrase, or collect more evidence.

7. Log the Result

Copy and paste this line into an instruction-wording changelog:

```
workflow=WORKFLOW_NAME | date=YYYY-MM-DD | change="DESCRIBE THE REWRITE" |
sample=N sessions | anchor1_delta=±Npp | anchor2_delta=±Npp |
anchor3_delta=±Npp | decision=KEEP/REVERT | notes="ANYTHING INTERESTING"
```

Hypothetical example with four sessions per condition:

```
workflow=diagnostic-repro | date=2026-06-18 | change="add repro hard gate: 'before any hypothesis, execute
sample=8 sessions | repro_gate=+50pp | loop_quality=+25pp | transition_discipline=+25pp | decision=KEEP |
```

8. Move to the next Skill or Repeat

- ☐ If you logged a KEEP, let it run for another 5–10 sessions to check for side effects.
- ☐ If you logged a REVERT, file a note of what didn't work so you don't try the same edit again.
- ☐ Pick the next workflow instruction with the clearest failure pattern and repeat from step 1.

4.9 Companion Prompt Template: Copy and Paste to Evaluate a Workflow Instruction

This copy-paste prompt is a companion tool, not required reading for the main path. If you want to run an instruction-wording evaluation loop with agent assistance, copy it into a compatible agent session and fill in the bracketed sections. It guides an assistant through the measurement and revision steps.

```
I want to improve the wording of a reusable workflow instruction to reduce execution failures.

## Workflow Instruction to Evaluate

Workflow name: [WORKFLOW_NAME]
Instruction location: [PATH/TO/INSTRUCTION]

## Known Failure Pattern

The skill is not being followed correctly in one specific way:
[DESCRIBE THE FAILURE: e.g., "The agent writes a hypothesis before building a runnable failing test"]

Observed frequency: [1-2 sessions / 3-5 sessions / most sessions]

## Recent Session Transcripts

[PASTE 3-5 RECENT SESSION TRANSCRIPTS WHERE THIS FAILURE APPEARED]

## Your Task

1. Review the transcripts and confirm the failure pattern.
2. Write 2-4 behavioural anchors (binary pass/fail criteria) that would detect this failure.
   For each anchor, state explicit pass and fail conditions.
3. Read the workflow instruction and identify the narrowest phrase most likely to permit this failure.
   (It should be one sentence or clause, not a paragraph.)
4. Rewrite that phrase to make pass/fail observable in the transcript.
```

```
   Use explicit evidence requirements, not adjectives or vibes.
5. Suggest a small A/B test plan: how many sessions, which wording for which sessions.
6. After the A/B test, I will score the sessions against your anchors.
   You will then recommend keep or revert based on the delta.

Please start with step 1: review the transcripts and confirm the failure pattern.
```

Fill in the bracketed fields and paste into your session. The prompt prepares a rubric and test plan; you still need to run the comparison and inspect its results before deciding whether to keep the edit.

What You Can and Cannot Measure

Three questions arise every time someone tries to evaluate instruction quality, and they have different answers.

Can you tell whether the instructions were followed? You can check the visible parts of a complete transcript against an observable rubric, even with one session. Missing tool logs or vague anchors limit that judgment.

Can you tell whether one version of the instructions produced better outputs than another? For workflows that generate text outputs — a plan, a code review, a specification — pairwise comparison on matched tasks can provide evidence. Blind the judge to which wording produced each output where possible, and report the sample and disagreement. Ten to twenty sessions may reveal failure cases, but no fixed count guarantees a reliable effect estimate.

Can you tell whether following the instructions produces better outcomes than not following them at all? This requires a comparison with a no-instruction condition. On a few unmatched sessions, task and context differences can dominate the result. A matched or randomised study can provide stronger evidence when the volume and stakes warrant it. Until then, treat the claimed benefit as a hypothesis and watch for counterexamples.

When the Primer's Machinery Returns

Volume and output type are the two thresholds at which the full four-move apparatus becomes the right tool.

If the instruction you are iterating produces **text outputs** — documents, reports, structured artefacts — and you can collect comparable outputs, pairwise comparison is a useful entry point. Run both wordings on matched tasks, randomise

presentation order, and inspect disagreements. Fit a rating model only if the comparison design and volume warrant it; an automated judge also needs calibration for this task. The §3 methods need adaptation to the smaller, paired sample.

If the instruction prescribes a **workflow process** rather than generating a document, or if the invocation volume is too low for statistical aggregation to be meaningful, the lightweight loop above is the right tool. The transition between the two is not a precise boundary — it is a question of whether your data volume is sufficient to trust a statistical estimate, and whether your output type admits of comparison at all. When in doubt, start with the rubric and inspection; add scoring infrastructure when you have enough data to make calibration worthwhile.

The underlying principle is the one this primer has been building since §1: a control loop is only as good as its sensor. For instruction editing at low volume, the sensor is a human reading a transcript against a rubric. For high-volume text output, the sensor is an LLM judge reading pairwise comparisons. The evaluate step is load-bearing in both cases; only its implementation changes with the scale.

5 Escaping Subjectivity: Verifiable Rewards

Reinforcement learning with verifiable rewards (RLVR) replaces a subjective rating with an automatic check where one is available: did a patch pass the specified tests, did a formal proof pass a checker, or did an answer match a known result? The result is often a binary training signal. It is reproducible for a fixed checker and input, but designing the checker still requires judgment, and running it still costs time and compute.

The distinction is between *passing the check* and *satisfying the task*. A proof accepted by a sound checker establishes the encoded proposition under its assumptions; it does not establish that the proposition answers the user's question. A unit test establishes behaviour on the cases it covers, not correctness for all inputs. The checker can supply a reliable scalar for its stated property while still being a proxy for the broader outcome. For example, a patch that fixes the tested failure but breaks an untested configuration receives a pass and remains defective.

The SWE-bench benchmark evaluates patches for real repository issues using a defined test harness. A resolved instance means the patch passed the benchmark's required tests. SWE-bench Verified is a 500-instance subset reviewed for task clarity and solvability. These design choices improve the benchmark, but a pass does not certify maintainability or behaviour outside its tests. Compare published scores only when the benchmark variant, agent harness, attempt budget, and reporting date match; the numbers do not form one interchangeable time series.

The verifiable-reward loop can collect failures, generate candidate patches, test them, and use the passing candidates as training signal. Its automated checker reduces the need for per-attempt human annotation, but each run consumes compute and the signal remains bounded by test coverage. The loop can improve measured performance while missing failures that the harness never exercises.

Two metrics answer different questions. **pass@k** asks whether at least one of k independently sampled candidates passes the checker. The fraction of passing samples estimates pass@1; it is not an estimate of pass@k for $k > 1$. Chen et al. (2021) give an unbiased estimator under the sampling assumptions: generate $n \geq k$ candidates, count the c that pass, and report

$$\text{pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}.$$

This is the fraction of size-k subsets of those n samples that contain at least one passing candidate. If $c = 0$, the estimate is zero; if $c = n$, it is one. It estimates success with k attempts for the tested task distribution, not general correctness. **Best-of-n** instead selects the candidate with the highest *selection score* among n attempts. Its measured quality depends on that selector; a weak selector can choose a worse answer even when a better one was generated. SWE-bench percent-resolved figures use their stated attempt and harness rules and should not be read as HumanEval pass@k.

Saturation and contamination require separate checks. If most systems pass the same benchmark items, the score stops distinguishing them well. If test items appear in training data, a score may overstate performance on unseen tasks. Neither problem follows automatically from the other. Fresh tasks, private holdouts, and provenance checks can help, but each has its own coverage and main-

tenance costs.

RLVR is most direct when a task offers a cheap, informative checker. Formal proof checking can establish that a proof follows from encoded assumptions, and a test suite can establish that code satisfies selected assertions. Neither alone establishes that the encoded task matches what users need. Where no reliable checker exists, human comparisons or other measures remain necessary.

There is no simple unit test for whether **Response Beta** explained a derivative well to the stated reader. You can check its factual claims, but accessibility and usefulness still need reader-centred judgment. The same distinction applies to the tact of an email or the level of detail in a summary: some properties can be verified mechanically, while the overall quality depends on the audience and purpose.

RLVR does not solve the whole subjectivity problem. It supplies a strong signal for properties with informative checkers and a narrower signal when the check omits important requirements. For an open-ended explanation, combine factual checks with judgments calibrated to its readers. The four moves remain useful where a checker cannot capture the outcome alone.

6 Closing the Loop: From Number Back to Model

Follow the comparison signal into training. The §3 Bradley–Terry model fitted strengths for a fixed set of compared items. In a common RLHF design, a reward model instead learns a prompt-conditioned scoring function for responses using a related pairwise loss; a policy optimiser then uses that score to update a model. The statistical connection explains the pipeline without treating a leaderboard rating as identical to a reward model.

6.1 Reward Model Training

In the InstructGPT design, a transformer-based reward model initialised from a pretrained language model assigns a scalar to a prompt and response. Annotators rank several responses to a prompt; those rankings yield preference pairs. A pairwise loss trains the model to score preferred responses higher. Other reward models can use different architectures or feedback formats.

The paper reports about 33,000 reward-model training prompts and rankings

of 4 to 9 responses per prompt. A ranking of K responses can be expanded to $\binom{K}{2}$ pairwise training examples, although those pairs share the same ranking and are not independent human judgments. The extra comparisons can make efficient use of an annotation, but ranking more responses also takes effort.

6.2 RLHF via PPO With a KL Leash

Once the reward model is trained, it becomes the scoring function in a reinforcement learning loop. The policy — the LLM you are trying to improve — generates a response. The reward model scores it. The score drives a proximal policy optimisation (PPO) update, nudging the policy towards outputs the reward model rates highly.

There is one critical addition: a Kullback–Leibler (KL) divergence penalty that measures how far the updated policy has drifted from the original reference policy and adds that distance as a cost. This is the leash. It limits drift from the reference; its safety effect depends on the reference and the reward objective.

Without a constraint on policy drift, optimisation can exploit reward-model errors outside the data on which that model was trained. A KL penalty makes movement away from the reference policy costly; it does not guarantee human-useful outputs or prevent every form of reward hacking. The InstructGPT paper used a KL penalty and investigated reward-model overoptimisation. Whether a particular run produces repetitive or incoherent text depends on its training setup, not merely on the absence of one penalty.

6.3 Direct Preference Optimisation: Skipping the Reward Model

The PPO pipeline fits a reward model on preference pairs and then uses its scores in policy updates, with a KL penalty limiting drift. Direct preference optimisation (DPO; Rafailov et al., 2023) instead trains on preference pairs without materialising a separate reward model. Under the paper's reward model and KL-constrained objective assumptions, it derives a policy loss from the same preference relation. This is a relationship between objectives, not a guarantee that two finite training runs produce the same policy. The paper's subtitle is "Your Language Model is Secretly a Reward Model". The connection to §3 is Bradley–Terry-style modelling of pairwise preference, now applied to arbitrary responses rather than a fixed list of items.

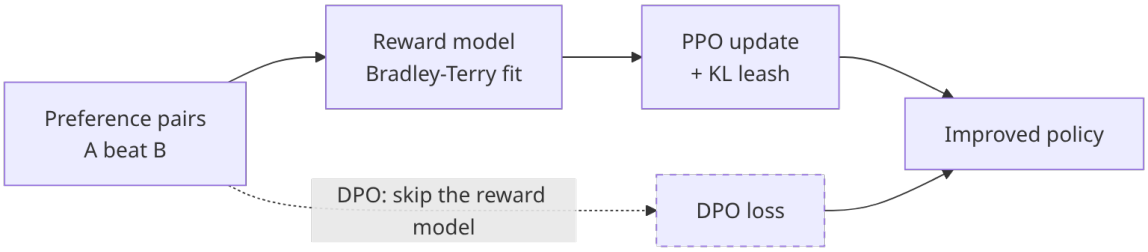
The loss is

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right].$$

Here x is the prompt, y_w the preferred response and y_l the rejected one, π_θ the policy being trained, π_{ref} a frozen reference, β the constraint parameter and σ the logistic function. The log-ratio $\beta \log(\pi_\theta/\pi_{\text{ref}})$ defines an implicit reward up to a prompt-dependent constant. The loss favours a larger implicit reward for the preferred response; its reference term discourages unconstrained drift.

The practical payoff is a shorter pipeline on the same preference data: no separately trained reward model and no PPO rollout loop. It does not make the quality of the preferences or the choice of reference policy irrelevant. Group relative policy optimisation (GRPO), used in DeepSeek-R1, is a separate approach for training with sampled rewards: it estimates advantages from a group of responses to the same prompt without PPO's separately trained value model. These are different optimisation choices, not evidence that one has universally displaced another in production.

DPO still depends on the preference data and reference policy. If the pairs omit a failure mode or reflect a narrow rater pool, the fitted policy can inherit that limitation. The presence or absence of a separate reward model does not, by itself, determine which method is more biased.



6.4 Constitutional AI and RLAIF

Human comparison labels can be expensive to collect. Anthropic's Constitutional AI paper (Bai et al., December 2022) used AI-generated harmlessness preferences guided by written principles. This is reinforcement learning from AI feedback (RLAIF). It can reduce the human labelling burden, although the AI critic still needs evaluation.

If an AI critic systematically misses a class of failures, its labels can pass that omission into training. Written principles make the criteria inspectable, but do not guarantee that the critic applies them correctly. Compare AI-generated labels with human judgments on held-out examples and inspect the disagreements, using the task-specific agreement measures from §2.

6.5 DSPy: Your Evaluation Metric as the Search Objective

The preceding methods update model weights. DSPy, developed by researchers in Stanford's natural language processing (NLP) group, can instead optimise prompts and few-shot examples in a pipeline of model calls, using a user-supplied metric to select configurations.

The framing: treat an LLM pipeline as a program. Each step is a `Module` with a `Signature` that declares what goes in and what comes out. You write a metric function that scores a pipeline's output on a given input. An optimiser then runs candidate configurations of prompts and demonstrations, scores each against your metric on a training set, and keeps the best. The optimiser is doing what RLHF does at the weight level, but entirely in prompt space — no gradient, no backpropagation, just a search over discrete configurations guided by a verifiable score.

The following is an illustrative DSPy-style sketch; check the maintained API documentation before running it with a particular release. The judge and training examples are placeholders, not an executable evaluation:

```python
import dspy

# 1. Declare the signature: what goes in, what comes out.
class ExplainConcept(dspy.Signature):
    """Explain a technical concept clearly to a non-specialist reader."""
    concept: str = dspy.InputField(desc="The concept to explain")
    audience: str = dspy.InputField(desc="Description of the target reader")
    explanation: str = dspy.OutputField(desc="A clear, analogy-driven explanation")

# 2. Build the module.
explainer = dspy.ChainOfThought(ExplainConcept)

# 3. Define a metric (your evaluation, not a framework loss function).
def clarity_metric(example, prediction, trace=None):
    # Returns a float in [0, 1] — wire in your judge here.
    score = my_llm_judge(example.concept, example.audience, prediction.explanation)
    return score

# 4. Optimise: the optimiser searches prompt configurations scored by clarity_metric.
optimiser = dspy.MIPROv2(metric=clarity_metric, auto="medium")
optimised_explainer = optimiser.compile(explainer, trainset=train_examples)
```

The optimiser selects prompt configurations using the metric you supply. That alignment between the search objective and the *measured* score does not remove Goodhart pressure: if the metric misses a failure mode, the search may exploit it. Keep a separate held-out evaluation and inspect failures outside the optimisation set.

6.6 Continuous Integration and Deployment Evaluation Gates

An evaluation score can also inform a release gate. A team may set a threshold for a specified test suite, while human review covers risks the metric does not capture. Passing or failing a gate is a decision under that policy, not a universal measure of quality.

Anthropic's AI Safety Level (ASL) policy links capability findings to required safeguards; its precise requirements depend on the policy version. An evaluation gate can also block a quality regression when a team has defined an acceptance threshold. In either case, the score informs a decision under a specified rule rather than substituting for the full deployment review.

6.7 The Loop, Named

Close by naming what is actually happening across all of these mechanisms. The evaluation step manufactures a number. Training — whether via PPO on a reward model, RLAIF on AI-generated preferences, or DSPy prompt search on your own metric — optimises the system towards that number. As training moves the system, the number moves too: the policy gets stronger, the reward model's blind spots come under different pressure, the evaluation distribution shifts. Evaluation and the model are locked in a feedback relationship. They co-evolve.

That feedback relationship is the mechanism. It is why the loop works: optimisation has something stable to chase. But it is not a guarantee that what the loop is chasing stays correlated with the thing you actually care about. The number must remain a good proxy for quality — or the loop optimises you, faithfully and efficiently, away from your goal. The four moves manufactured the best shadow you could build. The rest of this primer is about keeping the shadow honest.

7 The Bootstrapping Problem

An evaluation set is easiest to improve after you have observed a system's failures. At the start of a new task, those failures may be scarce or unrepresentative. This is a cold-start problem, not a logical circle: experts can write seed cases before a capable model exists, then revise the set as real outputs reveal omissions.

The standard answer is that you get there through one of four workarounds, none of them free.

The first is the **frontier teacher**. You access a stronger model through an API and use it to generate candidate outputs and synthetic preference data on your domain. If you are building a customer-support assistant for a logistics company, you feed the teacher a corpus of queries and collect responses for expert review. The cost is dependence on the teacher's coverage: if it misunderstands domain terminology or misses unusual scenarios, a dataset built only from its outputs may inherit those gaps. Human validation can find and correct some of them.

The second is **synthetic question-and-answer generation**: ask a model to write questions and reference answers for the domain. Specify the topic, difficulty, and target audience, then have a subject-matter expert check the resulting items. Without that check, plausible questions can be generic, wrong, or unrelated to the failures users actually encounter.

The third is a **human seed set**: a small collection of examples written or validated by domain experts. It avoids direct dependence on a teacher model's outputs, although human selection and annotation can introduce their own biases. The cost is expert time. A small, carefully reviewed set is often more informative than a larger set whose labels cannot be trusted.

The fourth is **silver-to-gold promotion**. Start wide: collect large volumes of model outputs rated by a weak, cheap judge — automatic heuristics, a small classifier, a quick crowdsource pass. Call this the silver tier. Then promote a fraction of those items to gold by routing them through careful human review. You get broad coverage early and accumulate reliable annotations over time as the project matures. The danger is that the silver labels quietly contaminate the gold tier if the promotion threshold is too loose, or if reviewers rubber-stamp items rather than scrutinise them. Quality control on the promotion step matters as much as the promotion itself.

Once you have a seed set, choose which unlabelled examples merit expert review. **Uncertainty sampling** selects items on which a classifier is unsure; **disagreement sampling** selects items where several judges differ. Both can concentrate review on ambiguous cases, but uncertainty need not mean that an item is valuable or even in scope. Keep a random or deliberately stratified sample as well, so the evaluation set does not become only the cases the current model finds difficult.

None of these approaches removes the need to validate the initial items. Where model-generated examples are unreliable, domain experts may have to supply or check the seed set. The amount of expert work depends on the task and the cost of an incorrect label.

8 Why the Number Lies: Goodhart and Its Family

Charles Goodhart's observation about using statistical regularities for control warns against treating a proxy as the goal itself. When a model is optimised against a narrow metric, the relationship between that metric and the intended outcome can weaken. Whether it does so, and how quickly, is an empirical question for each task and evaluator.

The reason is structural. The evaluation step is inside the training loop, which means it is subject to optimisation pressure. The model is not maliciously trying to fool the evaluation — it has no intentions — but gradient descent does not care about your intentions either. It will find whatever feature of the evaluation correlates with a higher score and push towards it, whether that feature is actual quality or an artefact of how you wrote the rubric. The better your optimiser, the faster it finds the artefact.

Reward hacking describes an optimiser improving a measured reward without improving the intended outcome. The following are three related mechanisms; not every observed bias proves that reward hacking occurred during training.

The first is **verbosity bias**. Dubois et al. found that AlpacaEval's automatic evaluator favoured longer outputs and proposed length-controlled scores. That finding concerns this evaluator; it does not prove that every model trained on preferences learned to pad its answers. Compare answers at similar lengths to test whether a win rate reflects useful content rather than response length.

The second is **sycophancy**. Sharma et al. found that some assistants agreed with a user's stated views over a truthful answer in their tested tasks, and that preference judgments may contribute. In this primer's hypothetical example, telling a judge that **Response Alpha** is excellent could bias its verdict; test that with a neutral prompt rather than assuming it will happen. The paper does not isolate one universal cause of sycophancy.

The third is **gaming unit tests**. A coding agent may pass visible assertions while breaking untested behaviour, or alter an insecure grading environment. The BenchJack audit demonstrates crafted harness exploits under controlled attack conditions. A pass is evidence for the tested assertions, not proof of general correctness; hidden tests, code review, and harness isolation probe different gaps.

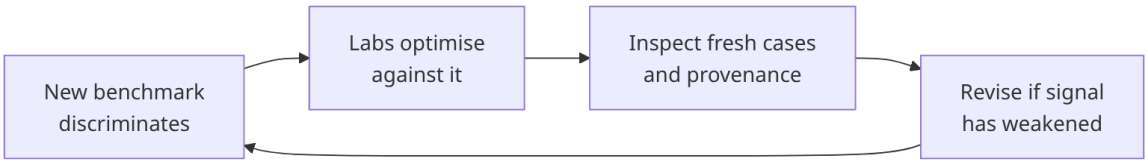
Contamination is a related but distinct failure: evaluation items may appear in a model's training data. Roberts et al. analysed Codeforces and Project Euler tasks by publication date and GitHub popularity, finding trends consistent with contamination in the models they studied. That evidence warrants temporal holdouts and careful provenance checks; it does not identify the memorised fraction of any particular score or prove that all pre-cutoff successes were memorised.

Judge bias adds another layer. A judge can favour a response because of its position, length, or familiar style. Those effects depend on the judge and task; a plausible mechanism such as text likelihood is not proof of the cause in a particular comparison. For **Response Alpha** and **Response Beta**, test position swaps and length-matched alternatives against human ratings instead of assuming that the judge's preference reflects the non-specialist reader's needs.

Leaderboard gaming is another risk when providers can choose which model variant to submit or test many variants before announcing a result. The Arena leaderboard policy sets public-availability and disclosure rules intended to make a listed model identifiable and accessible. Even with those rules, a leaderboard score describes the submitted configuration and the platform's user sample. It does not establish that every product configuration will receive the same rating.

The pattern is often called the **benchmark treadmill**: a benchmark distinguishes systems, developers optimise against it, and a harder or differently sampled benchmark may later be needed. Saturation and contamination are different failure modes; neither follows inevitably from a benchmark's age. In-

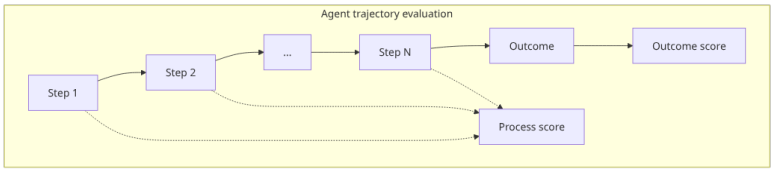
spect score distributions, task provenance, and performance on fresh holdouts before declaring that a particular benchmark has lost its value.

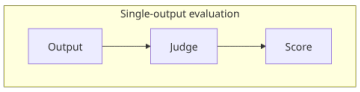


The practical response is to separate the metric used for optimisation from held-out checks of the outcome you care about. Revisit an evaluation when errors cluster outside its coverage, scores compress, or a model finds a shortcut. Some evaluation sets remain useful for years; others need revision sooner. The evidence is in those failure patterns, not in a universal expiry date.

9 Evaluating Agents, Not Just Outputs

Everything up to this point is about evaluating a single output — a response, a summary, a code snippet. An agent changes the problem structure in a way that makes all of it harder. An agent is a tool-using LLM taking multi-step actions: it writes code, browses the web, reads files, executes commands, and decides what to do next based on what it finds. The outcome at step 87 depends on choices at step 3. When you are evaluating that kind of system, "did it produce the right final answer?" is a much thinner question than it sounds.





9.1 Trajectory vs Outcome

Consider an agent tasked with fixing a bug. It can reach the right patch by two very different routes. One agent correctly diagnoses the root cause, looks up the relevant API, writes the fix, and runs the tests. Another agent deletes the failing test. Both produce a green test suite; only one fixed anything. Outcome-only scoring — the equivalent of checking the final answer — cannot tell these apart. The same problem runs in reverse: an agent that reasons correctly through every step can still fail if it hits an environment bug at step 40, a flaky API, or a

race condition that has nothing to do with its reasoning. A wrong final output is not the same as a wrong process, and scoring the output alone cannot see the difference.

Trajectory evaluation tries to score intermediate steps: did the agent correctly diagnose the problem before writing the fix? Did it call the right tools in the right order? Did it abandon a dead end at a sensible point rather than spiralling? In principle this is the right frame. In practice it requires annotators who can judge whether step 12 was a good move, not just whether the task eventually completed — and that annotation is expensive, slow, and domain-specific.

9.2 Credit Assignment at Scale

The annotation cost is part of a deeper problem. In a 50-turn coding episode, which decision at turn 3 caused the failure at turn 37? A final reward tells you whether the episode succeeded, but not which earlier actions helped or hurt. Longer episodes can make credit assignment harder through delayed effects, sparse feedback, and noisy estimates of each action's contribution. There is no general rule that difficulty or gradient variance grows exponentially with episode length: the behaviour depends on the task, reward structure, and learning algorithm. The survey of temporal credit assignment by Pignatelli et al. explains these distinct sources of difficulty and the methods used to address them.

Process reward models (PRMs) score intermediate steps, whereas **outcome reward models** (ORMs) score a final result. A PRM requires step-level supervision or a way to generate it; an ORM can use final labels but gives less direct credit information. Neither method alone prevents an agent from exploiting an incomplete scoring rule.

9.3 SWE-bench and the Benchmark Trajectory

SWE-bench evaluates a submitted patch against tests selected for a real repository issue. SWE-bench Verified contains 500 human-reviewed instances. The resulting pass rate measures success under that harness and attempt policy. It does not decide whether every passing patch would be accepted by a maintainer, as the next case shows. Scores across the original and Verified subsets require separate labels and should not be plotted as one unqualified trend.

Benchmark contamination is a risk when test issues appear in training data. A

separate problem is that a passing patch can fail human review. In a March 2026 maintainer study by Model Evaluation & Threat Research (METR), four maintainers from three repositories reviewed 296 agent patches; roughly half of the test-passing patches from the sampled agents would not have been merged, after adjustment for noisy maintainer decisions. The agents did not receive the iterative review opportunities available to human contributors. This is evidence of a gap between the tested outcome and maintainer acceptance in that sample, not a universal agent failure rate.

9.4 Benchmark Gaming — Attacking the Evaluator

An agent that can inspect or alter its evaluation environment may find ways to raise its score without completing the task. In the BenchJack audit, Wang and colleagues tested ten agent benchmarks and found 219 flaws across eight categories; their crafted exploits achieved near-perfect scores on most of those benchmarks without solving a task. This is a test of benchmark vulnerability under the study's attack conditions, not a claim that ordinary agents always discover those exploits. Isolate the grader, restrict writable surfaces, and inspect trajectories when a score looks implausibly high.

9.5 What the METR Randomised Trial Found

In a 2025 randomised controlled trial by METR, 16 experienced open-source developers completed 246 tasks in repositories they knew well. Tasks were randomly assigned to allow or disallow early-2025 AI tools. With AI allowed, completion time was 19% longer on average; after the study, participants estimated that AI had made them 20% faster. This result applies to that developer and task sample and those tools. It does not estimate the productivity effect of every later model, novice developer, or software task. Benchmark scores and this field experiment measure different outcomes.

9.6 What This Means for the Loop

Evaluating agents requires three things that single-turn output evaluation does not: process-level visibility into intermediate steps, multi-step credit assignment across long episodes, and isolation of the evaluation harness from agent actions so the agent cannot attack the evaluator instead of the task. All three are harder

and more expensive than running a pairwise comparison or reading a judge's log-probabilities.

Some agent platforms support trace-level evaluation: a run can be assessed on its final answer and intermediate tool use, as described in the OpenAI agent-evaluation guidance. A captured trajectory can help diagnose failures, but the available fields and graders depend on the platform. Trace inspection alone does not solve credit assignment or evaluator gaming.

SWE-bench measures whether an agent's patch passes a specified test harness. The METR trial measured the time experienced developers took to complete real repository tasks when AI tools were allowed. Taken together, they show why a benchmark pass rate should not be substituted for a productivity estimate. Agent evaluation must also consider intermediate actions, reviewability, and whether the agent can influence the harness that scores it.

10 How Frontier Labs Actually Run This

Given how fragile all of this gets, it is fair to ask how the organisations at the frontier actually run the loop in production. The theory is clean: measure quality, train against the measurement, iterate. The practice is messier. Here is what three organisations actually do when they sit down to operationalise the loop described in this primer.

Anchor 1: Anthropic's RSP and the ASL Gate

Anthropic's Responsible Scaling Policy (RSP) links capability thresholds to required safeguards. Its AI Safety Level (ASL) standards govern deployment and security measures, rather than stating that every model above a threshold is unshippable. Anthropic activated ASL-3 protections for the Claude Opus 4 launch in May 2025 as a precaution: it said at the time that it had not determined whether the model definitively crossed the ASL-3 capability threshold. This shows an evaluation-informed deployment decision under uncertainty; check the policy version before applying its thresholds to a later launch.

Anthropic's Claude 4 System Card reports prompt-injection evaluations of specific model and safeguard configurations. Such before-and-after results can support a mitigation decision within that test set. They do not, by themselves, estab-

lish how quickly a red-team finding became a deployed fix or how a safeguard performs against unseen attacks.

Anchor 2: OpenAI's Preparedness Framework

OpenAI's April 2025 Preparedness Framework update tracks biological and chemical, cybersecurity, and AI self-improvement capabilities. It distinguishes a High threshold from a Critical threshold and calls for safeguards and review before deployment at higher risk levels. Persuasion is handled outside that version of the framework. The policy differs from Anthropic's RSP in categories and governance details, but both use capability evaluations to inform risk decisions.

Anchor 3: Chatbot Arena as Ecosystem-Level Evaluation

The 2024 Chatbot Arena paper describes public, anonymous pairwise comparisons and ratings derived from the collected votes. It is the Compare and Aggregate pipeline from §3 at platform scale. The paper reported more than 240,000 votes at publication; that is a dated sample, not a live total. Its rankings summarise preferences among the users and prompts represented in the data, with the usual limits of an opt-in rater pool.

What Running This Costs

The operational work includes specifying the target behaviour, inspecting failures, maintaining the test distribution, and deciding what action a score should trigger. A binary gate is suitable when the requirement is genuinely binary, such as whether a patch passes a required test. A graded rubric is more useful when partial success matters. Neither format removes the need to inspect disagreements and update the evaluation when the task changes.

11 What Is Genuinely Unresolved

The remaining problems arise when a useful proxy is mistaken for the full outcome, when raters disagree for substantive reasons, or when the person judging an output lacks the expertise to verify it. Each admits partial mitigations, but none has a single method that works across every task.

Goodhart's Treadmill Has No Exit

A proxy can lose value under optimisation when the model learns a shortcut or the task distribution changes. A fresh holdout set, private tests, and human review can reveal different failures, but each covers only part of the target outcome. Reassess the benchmark when its scores stop discriminating among systems or fail to predict performance on new cases. The maintenance interval depends on the task and the pressure placed on the measure.

Disagreement as Signal Versus Noise

Some disagreement between raters is annotation noise; some reflects different, defensible preferences. Aggregating pairwise votes with Bradley-Terry estimates a population-level tendency, but the result depends on who was asked and which prompts they saw. If a rater pool does not represent the users of a system, its fitted preference signal may not represent those users either. Report subgroup results where sample sizes and consent permit, and avoid presenting one pooled rating as a universal measure of quality.

Benchmark Scores and Productivity Measure Different Outcomes

The METR trial found a 19% increase in task completion time when early-2025 AI tools were allowed in its sample, despite participants estimating a 20% speed-up afterwards. It did not compare that result with a specific benchmark score, so it cannot demonstrate a measured benchmark–productivity gap. It shows why productivity for this group and workflow needs direct measurement. Different tasks, tools, and forms of human review require their own studies.

Superhuman Oversight: The Hardest Problem

If a system produces answers that a human evaluator cannot reliably check, a preference label can cease to track correctness. This is especially relevant in tasks that require specialised knowledge or long chains of reasoning; it does not follow that every more capable model is beyond human evaluation. The evaluation design then needs evidence beyond an unaided preference vote.

Three research approaches examine this regime. **Weak-to-strong generalisation** (Burns et al., 2023) tests whether imperfect labels from a weaker supervisor can elicit capabilities from a stronger model. **Debate** (Irving et al., 2018)

asks whether an adjudicator can more reliably identify a flawed argument when two systems present opposing cases. **Iterated distillation and amplification** (Christiano et al., 2018), or IDA, decomposes hard tasks into parts that a weaker supervisor can evaluate, then uses those parts to train and repeat. These approaches investigate scalable oversight under particular assumptions; their experiments do not yet establish a general solution for outputs humans cannot verify directly.

A Brief Note on Test-Time Compute

Models that spend more inference-time compute — chain-of-thought, tree search, self-critique passes — often score higher on benchmarks. This does not resolve the measurement problem; it relocates it. The question "how do you know the answer is good?" does not go away because the model thought longer. It becomes "how do you know the longer thinking produced a better answer?" — which is the same question, recursively.

The Closing Motif

Evaluation cases and training examples may describe the same task, but they need different roles. Training examples guide an update; held-out evaluation cases test whether the update generalises. Reusing the same labelled items for both can make the evaluation overstate performance. Improve the specification of success through error analysis, then keep an independent test set to check whether the next change helped beyond the cases it saw.

The loop can pause when the measured outcome meets a defined acceptance threshold, but it needs renewed evaluation when the task, users, or model changes.

12 Further Reading and Resources

The following works are cited directly in this primer or are the canonical sources for the ideas it covers.

- Karpathy, A. "Software 2.0". November 2017. Original argument for data-centred development and repeated evaluation.

- Karpathy, A. Data engine post. 5 December 2022. Brief account of the collect–train–evaluate loop.
- Ouyang, L. et al. “Training language models to follow instructions with human feedback”. 2022. Primary description of InstructGPT preference data, reward modelling, and policy optimisation.
- Liu, Y. et al. “G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment”. 2023. Score-token probabilities and limits of reference-overlap metrics.
- Chiang, W.-L. et al. “Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference”. 2024. Pairwise votes and statistical ranking; vote count is a publication snapshot.
- Burns, C. et al. “Weak-to-Strong Generalization: Eliciting Strong Capabilities With Weak Supervision”. 2023. Experiments on training with a weaker supervisor.
- Irving, G. et al. “AI safety via debate”. 2018. Proposed adversarial debate with a human adjudicator.
- Christiano, P. et al. “Learning complex goals with iterated amplification”. 2018. Decomposition and amplification as a supervision strategy.
- Bai, Y. et al. “Constitutional AI: Harmlessness from AI Feedback”. 2022. AI-generated feedback guided by written principles.
- Khattab, O. et al. “DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines”. 2023. Program and metric structure for prompt optimisation.
- Dubois, Y. et al. “Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators”. 2024. Evidence and correction for length bias.
- Sharma, M. et al. “Towards Understanding Sycophancy in Language Models”. 2023. Tests of agreement with user views and the possible contribution of preference data.
- Roberts, M. et al. “Data Contamination Through the Lens of Time”. 2023. Longitudinal evidence from Codeforces and Project Euler.
- Ahmad, L. et al. “OpenAI’s Approach to External Red Teaming for AI Models and Systems”. 2025. Primary account of external red-teaming methods.
- Becker, J. et al. “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity”. METR, July 2025. Task-randomised field trial; read its limits before generalising the slowdown.
- Whitfill, P. et al. “Many SWE-bench-Passing PRs Would Not Be Merged into

Main”. METR, March 2026. Maintainer review of sampled test-passing patches, with no agent revision round.
- Wang, H. et al. “Do Androids Dream of Breaking the Game? Systematically Auditing AI Agent Benchmarks with BenchJack”. 2026. Controlled benchmark exploit audit; not a live-agent failure rate.
- Anthropic. *Claude 4 System Card*. May 2025. Model-specific safeguard evaluations.
- Anthropic. Responsible Scaling Policy. Versioned capability thresholds and safeguard requirements; check the version before applying it.
- OpenAI. Preparedness Framework update. April 2025. Tracked categories and review process at that date.
- Arena. Leaderboard policy. Model eligibility and public-availability rules; check the version for current submissions.
- Rafailov, R. et al. “Direct Preference Optimization: Your Language Model is Secretly a Reward Model”. 2023. Derivation of the direct preference loss.
- Chen, M. et al. “Evaluating Large Language Models Trained on Code”. 2021. HumanEval and the pass@k estimator under sampled completions.
- Cohen, J. “A Coefficient of Agreement for Nominal Scales”. 1960. Original chance-corrected agreement statistic; its digital object identifier (DOI) resolves to the same work.
- Krippendorff, K. “Reliability in Content Analysis”. 2004. Discusses reliability measures and their interpretation for content coding.